

SKEL 4223: Digital Signal Processing *with* MATLAB

CHAPTER 3

LTI SYSTEM

Dr. Musa Mohd Mokji

1.0 Difference Equation

```
x = [1 2 3 1 0 -2 3];  
x = [0 x 0];  
  
N = length(x);  
y = zeros(1,N);  
  
for n = 2:N  
    y(n) = x(n)+2*x(n-1);  
end  
  
x, y
```

1.1 Introduction

Difference equation is an equation that represents an LTI system. The general equation of the difference equation is as below:

$$y[n] = \sum_{k=-\infty}^{\infty} b_k x[n - k] + \sum_{k=-\infty}^{\infty} a_k y[n - k]$$

In this example, let's consider the following difference equation. We will write a code to compute the output $y[n]$ given the input signal $x[n]$.

$$y[n] = x[n] + 2x[n - 1]$$

```
x = [1 2 3 1 0 -2 3];
```

1.2 Zero Padding

Because the difference equation involves past input values, a value before the first sample of x is needed. Since it is known that the previous value is zero, thus zero padding method is used. In this example, there is only 1 previous value, thus pad x with 1 preceding and trailing zero.

This can be done by creating a new vector by adding 0 before and after the x , as shown below. If the preceding and trailing zeros are many, it is more appropriate to use function `zeros()`.

```
xp = [0 x 0];
```

1.3 Implementing the DE

Since the original x was padded with zero, the starting n is shifted to how many zeros were padded.

```
N = length(xp);
y = zeros(1,N);

for n = 2:N
    y(n-1) = xp(n)+2*xp(n-1);
end
xp, y=y(2:end)
```

```
xp =
    0     1     2     3     1     0    -2     3     0

y =
    1     4     7     7     2    -2    -1     6
```

2.0 Difference Equation with Feedback

Now, let's consider another example of DE but with a feedback:

$$y[n] = x[n] - 2x[n - 1] + 3y[n - 2]$$

Like the previous example, the input signal will need to be padded with zeros. This time, 2 zeros will be padded at the preceding and trailing of the x . This is because the highest amount of delay occurs on the DE is 2.

```
x = [1 2 3 1 0 -2 3];
xp = [0 0 x 0 0];
N = length(x);
y = zeros(1,N);

for n = 3:N
    y(n) = xp(n)-2*xp(n-1)+3*y(n-2);
end
xp, y=y(3:end)
```

$x =$

0 0 1 2 3 1 0 -2 3 0 0

 $y =$

1 0 2 -5 4 -17 19 -57 57

3.0 Difference Equation given Impulse Response

Difference equation can be described as the weighted sum of the input signal samples. For FIR system, the weight is the impulse response of the system. Given impulse response $h=[1 \ 2 \ 1]$, x need to be padded with 2 preceding and trailing zeros where delay amount is length h minus 1.

```
h = [1 2 1];
x = [1 2 3 1 0 -2 3];
xp = [0 0 x 0 0];
y = zeros(1,length(x));

for n = 3:length(x)
    y(n) = h(1)*xp(n)-h(2)*xp(n-1)+h(3)*xp(n-2);
end
xp, y=y(3:end)
```


$x =$

0 0 1 2 3 1 0 -2 3 0 0

 $y =$

1 0 0 -3 1 -1 7 -8 3

4.0 Convolution

```

h = [1 2 1 1];
x = [1 1 1 2 2 2];
M = length(h)-1;
N = length(x)-1;

xp = [zeros(1,M) x zeros(1,M)];
y = zeros(1,M+N+1);
yk = zeros(1,length(h));

for n=M+1:2*M+N+1
    for k=1:M+1
        yk(k) = h(k)*xp(n+1-k);
    end
    y(n-M)=sum(yk);
end
h, x, y
    
```

4.1 Introduction

In LTI system, difference equation is stored in a memory by storing the values of the weight a_k and b_k . For FIR system, only b_k is available. This weight presence of b_k without the a_k is the impulse response of the FIR system. Equation that consist only the weight b_k is then called convolution as the following.

$$y[n] = \sum_{k=-\infty}^{\infty} b_k x[n - k]$$

Example in Section 1.0 and 3.0 are the examples of convolution. In this section, the code will be generalized to accept any impulse response h and input signal x . Below are the input signal x and impulse response that will be used in this example.

```
h = [1 2 1 1];
x = [1 1 1 2 2 2];
```

4.2 Time Shifting

To generalise the convolution equation, all signal (x , h and y) are assumed to start at $n=0$. Thus, the equation can be written as below:

$$y[n] = \sum_{k=0}^M h[k]x[n-k] \quad \text{for } 0 \leq n \leq M+N$$

Then, to avoid indexing value less than 1:

- 1) All indices are added by 1
- 2) Signal x is shift by M sample through padding the signal x with M preceding and trailing zeros. Let x_p be the padded x , below is the modification to the convolution equation:

$$y[n] = \sum_{k=1}^{M+1} h[k]x_p[n+1-k] \quad \text{for } M+1 \leq n \leq 2M+N+1$$

Below is how the signal is padded with zeros.

```
M = length(h)-1;
N = length(x)-1;
xp = [zeros(1,M) x zeros(1,M)];
```

xp =

Columns 1 through 10

0 0 0 1 1 1 2 2 2 0

Columns 11 through 12

0 0

4.3 Using Loops for the Repetition Computation

Based on the convolution equation, there are two repetition computations. First is the summation of the range $1 \leq k \leq M+1$ and second is the computation on each n within the range of $M+1 \leq n \leq 2M+N+1$. Thus, there will be two loops, each for k and n .

```

y = zeros(1,M+N+1);
yk = zeros(1,length(h));
for n=M+1:2*M+N+1
    for k=1:M+1
        yk(k) = h(k)*xp(n+1-k);
    end
    y(n-M)=sum(yk);
end
h, x, y
    
```

$h =$

1 2 1 1

$x =$

1 1 1 2 2 2

$y =$

1 3 4 6 8 9 8 4 2

Try other values for the h and x , and observe the output. The code should work for any valid inputs.

5.0 Convolution using Matrix Operation

```
x = [2 3 1 6];
h = [1 4 7];
y = convolution(x,h);
```

```
function y = convolution(x,h)
    N = length(x)-1;
    M = length(h)-1;
    x = [zeros(1,M) x zeros(1,M)]; % Padding x with zeros
    [A,B] = meshgrid(0:M+N,0:M);
    q = A-B + M+1
    X = x(q)
    y = h*X
end
```


5.1 Introduction

Since matlab works great on matrix, lets formulate the convolution in matrix form. Basically, convolution involve shifting, folding, multiplication and sum operations. The multiplication and sum operation in convolution is simply a matrix multiplication as the following.

$$y[n] = \sum_{k=-\infty}^{\infty} h[k]x[n-k]$$

$$y = h \cdot X$$

where h is a row vector and X is an array of shifted signal x .

As a start, below are the signals that will be used in this example and the convolution process is perform by calling the function `convolution()`.

```
x = [2 3 1 6];
h = [1 4 7];
y = convolution(x,h);
```

5.2 The convolution() Function

This function accepts h and x as the input signals and return y as the result of the convolution. The function will first create array X (contain list of shifted signal x). Then, matrix multiplication is performed to complete the multiplication and sum operations.

All the three signals x , h and y are assumed to have a start time at $n=0$.

```
function y = convolution(x,h)
```

5.3 Create Matrix of Shifted Signal x

In convolution, nonzero y elements are computed from n_s to n_e . n_s is the sum of the lowest n of input signal x and impulse response h and n_e is the sum of the M and N . M is the highest n for h and N is the highest n for x . Thus, the formulation can be written as below:

$$y[n] = \sum_{k=0}^M h[k]x[n-k] \quad \text{for } 0 \leq n \leq M+N$$

$$y = h \cdot X$$

$$= [h(0), h(1), \dots, h(M)] \cdot \begin{bmatrix} x(0) & x(1) & \dots & x(M+N) \\ x(-1) & x(0) & \dots & x(M-1+N) \\ \vdots & \vdots & \ddots & \vdots \\ x(-M) & x(M+1) & \dots & x(N) \end{bmatrix}$$

$$= [h(p)] \cdot [x(q)]$$

Since MATLAB does not accept indexing value less than 1, above equation is modified as the following where indexing for h is add with 1 and indexing for x is add with $M+1$:

$$y = [h(p + 1)]. [x(q + M + 1)]$$

Below is how a meshgrid function is used to create matrix X by using q as the indexing for x .

```
N = length(x)-1;
M = length(h)-1;
x = [zeros(1,M) x zeros(1,M)]; % Padding x with zeros
[A,B] = meshgrid(0:M+N,0:M);
q = A-B + M+1
X = x(q)
```

q =

3	4	5	6	7	8
2	3	4	5	6	7
1	2	3	4	5	6

X =

2	3	1	6	0	0
0	2	3	1	6	0
0	0	2	3	1	6

Above shows that X has 6 columns and 3 rows. The column length is equal to the length of output y. In this case is where y has the range of $0 \leq n \leq 5$. Then, the array has 3 rows since the length of h is 3. On every row, x is shifted 1 sample to the right (delay by 1 sample).

5.4 Matrix Multiplication

By having the matrix X, which contain list of shifted x, the convolution can be perform directly by implementing matrix multiplication as below.

$$y = h * x$$

y =

2 11 27 31 31 42

end

6.0 Convolution via Function conv()

```

N = 500; n = 0:N-1;           % 500 samples
x = sin(0.01*pi*n)+0.1*cos(0.5*pi*n);

h1 = [1/4 1/4 1/4 1/4];      % Impulse response 1
h2 = [1/4 1/2 1/4];          % Impulse response 2
h3 = [1/4 -1/2 1/4];         % Impulse response 3
y1 = conv(h1,x);
y2 = conv(x,h2);
y3 = conv(x,h3);

plotsig(x,1,N), title('Input signal x[n]')
plotsig(y1,2,N), title('Filtering 1')
plotsig(y2,3,N), title('Filtering 2')
plotsig(y3,4,N), title('Filtering 3')

function plotsig(x,k,N)
    subplot(4,1,k), plot(x)
    ylim([-1.2 1.2]), xlim([0 N])
    xlabel('Time, n'), ylabel('Amplitude')
end
    
```

6.1 Introduction

From MATLAB library, convolution can be implemented using function `conv()`. Its simplest syntax accepts two inputs and return one outputs. To demonstrate the usage, filtering process to the following input signal x will be shown. 3 impulse responses will be used to filter the signal and compare their outputs.

$$x = \sin(0.01\pi n) + 0.1\cos(0.5\pi n)$$

Based on the above equation, signal x has two frequencies, which are 0.01π and 0.5π .

```
N = 500; n = 0:N-1;           % 500 samples
x = sin(0.01*pi*n)+0.1*cos(0.5*pi*n);

h1 = [1/4 1/4 1/4 1/4];      % Impulse response 1
h2 = [1/4 1/2 1/4];          % Impulse response 2
h2 = [1/4 -1/2 1/4];         % Impulse response 3
```


6.2 Using the conv() Function

Since convolution is a linear process, position of the input signal and impulse response to the input of the conv() function are interchangeable. In this example, the impulse response is set as the first input for the first filtering and set as the second input for the second filtering.

```
y1 = conv(h1,x);  
y2 = conv(x,h2);  
y3 = conv(x,h3);
```

6.3 Plotting the Result

It is observed that the first impulse response removed entirely the 0.5π frequency component of the input signal while the second impulse response is slightly attenuate the 0.5π frequency component. On the other hand, the third impulse response remove the lower 0.01π frequency component and retain the 0.5π frequency component.

```
plotsig(x,1,N), title('Input signal x[n]')
plotsig(y1,2,N), title('Filtering 1')
plotsig(y2,3,N), title('Filtering 2')
plotsig(y3,4,N), title('Filtering 3')

function plotsig(x,k,N)
    subplot(4,1,k), plot(x)
    ylim([-1.2 1.2]), xlim([0 N])
    xlabel('Time, n'), ylabel('Amplitude')
end
```

